PUBLIC

Code Assessment

of the Aave V4 Smart Contracts

March 25th, 2026

Produced for

Aave Labs

by

CHAINSECURITY

Contents

1 Executive Summary

Dear Adam Schoeman (CISO, Aave Labs), Emilio Frangella (SVP of Engineering, Aave Labs), Stani Kulechov (CEO, Aave Labs), dear Aave Labs team,

Thank you for trusting us to help Aave Labs with this security audit. Our executive summary provides an overview of the subjects covered in our audit of the latest reviewed contracts of Aave V4 according to Scope to support you in forming an opinion on their security risks.

Aave V4 introduces a lending protocol with a modular architecture centered around the Hub and Spoke model, designed to unify liquidity management across multiple markets while preserving isolation and configurability.

The most critical subjects covered in our audit are arithmetic precision and asset solvency. Security regarding all the aforementioned subjects is high.

The general subjects covered are liquidations design, functional correctness, and event handling.

In summary, we find that the codebase provides a high level of security.

Edgecase behaviors of the system have been highlighted in the Notes section.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but do not replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. Please note that issues which have not changed between Version 5 (`v0.5.7`) and this latest version (Version 9, `v0.5.11`) of the report have been omitted here. These issues are documented in the previous report. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	3
• Code Corrected	2
• Acknowledged	1

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The following contracts are in scope:

```
src/access/AccessManagerEnumerable.sol
src/hub/AssetInterestRateStrategy.sol
src/hub/Hub.sol
src/hub/HubConfigurator.sol
src/hub/libraries/AssetLogic.sol
src/hub/libraries/Premium.sol
src/hub/libraries/SharesMath.sol
src/libraries/math/MathUtils.sol
src/libraries/math/PercentageMath.sol
src/libraries/math/WadRayMath.sol
src/libraries/types/Roles.sol
src/position-manager/PositionManagerBase.sol
src/position-manager/PositionManagerIntentBase.sol
src/position-manager/NativeTokenGateway.sol
src/position-manager/SignatureGateway.sol
src/position-manager/libraries/EIP712Hash.sol
src/spoke/AaveOracle.sol
src/spoke/Spoke.sol
src/spoke/SpokeConfigurator.sol
src/spoke/TreasurySpoke.sol
src/spoke/instances/SpokeInstance.sol
src/spoke/instances/TreasurySpokeInstance.sol
src/spoke/libraries/EIP712Hash.sol
src/spoke/libraries/KeyValueList.sol
src/spoke/libraries/LiquidationLogic.sol
src/spoke/libraries/PositionStatusMap.sol
src/spoke/libraries/ReserveFlagsMap.sol
src/spoke/libraries/SpokeUtils.sol
src/spoke/libraries/UserPositionUtils.sol
src/utils/IntentConsumer.sol
src/utils/Multicall.sol
src/utils/NoncesKeyed.sol
src/utils/Rescuable.sol
```

The assessment was performed on the source code files inside the Aave V4 repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received. Please note that issues which have not changed between Version 5 (`v0.5.7`) and this latest version (Version 9, `v0.5.11`) of the report have been dropped. Refer to the previous report for a full picture.

V	Date	Commit Hash	Note
1	6 Oct 2025	dc31f9a4d54c0503093ef6939e6e8a8d2586709d	Initial Version

2	9 Oct 2025	14a824b258eb76b7a1ec86bd6cbe79da2539aed8	updated code
3	19 Nov 2025	91bb988323a635cdf67c1fb36d039306a8a6528b	third version
4	30 Nov 2025	6959e3219b5506bf2acae18551cbb2a68a5b8fba	v0.5.6
5	28 Jan 2026	31afa65a91f99ca7ec0437a1b438f65d3261d164	v0.5.7
6	4 Feb 2026	4ffe2a4ed272b707faea23176d48b349afc63358	AccessManager in scope
7	10 Feb 2026	c3b92877ec49c791e7eaaf77afe73eabd9bb7b1e	v0.5.9
8	12 Mar 2026	1486b5165b0ded213c5b6c1b52d046e3f221e5a4	v0.5.10
9	23 Mar 2026	cdacec509e4f848bff1a5556f503afa83eee3b79	v0.5.11

For the solidity smart contracts, the compiler version `0.8.28` was chosen.

2.1.1 Excluded from scope

Any contracts not explicitly listed in the scope section above are excluded from the assessment. Third party dependencies are assumed to behave according to their specification.

Deployment scripts and tests are also excluded from the scope.

2.2 System Overview

This system overview describes Version 2 of the contracts as defined in the Assessment Overview.

At the end of this report section, we have added subsections for each of the changes according to the versions.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Aave Labs offers Aave V4, which introduces a Hub - Spoke architecture that separates global liquidity accounting from user interactions. The Hub acts as a central vault that holds all assets and tracks the system's total liquidity and debt, while Spokes serve as gateways through which users supply, withdraw, borrow, repay, and interact with the protocol. It is expected that one or more Hubs get deployed per chain and that a variety of Spokes connect to these Hubs. This design aims at increasing modularity and flexibility.

2.2.1 Hub

The Hub is an immutable contract that holds all assets. The users of the Hub are the Spokes, and for every supported asset, the Hub keeps track of the amount added (supplied) by every Spoke, the available liquidity, the amount drawn (borrowed) by every Spoke, the amount *swept* by governance, and possibly the deficit (realized bad debt). In short, the Hub keeps a global view of the liquidity and the debt positions of Spokes for all assets. It also manages the interest rate strategies for each asset and handles interest accrual over time for each asset.

The Hub implements share-based accounting to keep track of the "added" and "drawn" amount for each Spoke.

2.2.1.1 Adding Liquidity

Spokes supply (*add* in Hub terminology) assets to the Hub and get *added shares*. The value of added shares is determined by the amount of assets in the Hub and the total existing supply of shares. The total amount of one asset in the Hub is the sum of the available liquidity, the amount *swept*, the deficit and the total owed amount of that asset. The share price for added shares is defined as the ratio between total assets and total added shares, and is strictly increasing over time. It can increase due to interest accrual, premium debt interest, and rounding in favor of existing shareholders when assets are added or removed. The total assets include an amount of 10^6 virtual assets and 10^6 *dead* shares, to prevent first depositor attacks and prevent share price inflation.

The `add()` function can be called by a Spoke to supply an asset. The Spoke must be enabled for the asset, and the Spoke's total added amount cannot exceed the Spoke's `addCap` for the asset. `add()` takes the asset amount as an argument, and mints the corresponding amount of shares, possibly rounded down. The asset is transferred to the Hub from an address specified by the calling Spoke. Liquidity can be withdrawn with the `remove()` function, which lets the Spoke specify an asset amount and a recipient. Shares belonging to the Spoke are burnt by converting the asset amount, possibly rounding up. The `remove()` call can fail because the requested liquidity is unavailable, either because it has been drawn or swept.

2.2.1.2 Drawing Liquidity

Spokes can borrow assets from the Hub (*draw* in Hub terminology). The amount Spokes owe is recorded as *drawn amount* (stored in debt shares) and *premium debt*. The *drawn amount* of debt is equal to the amount of drawn shares times the *drawn index*. The drawn index of the asset is a 27-decimal precision number which is linearly increased by the interest rate accrual since the last update starting from 1. When accrued often, the linear accruals are functionally equivalent to exponentially compounding accruals. The interest rate is queried from an *interest rate strategy* and depends on the utilization factor of the asset in the Hub. Since the interest rate is positive, the drawn index is strictly increasing over time.

Another component of the total owed amount is *premium debt*. This is a novel Aave V4 feature that allows charging higher interest rates when borrowing against riskier collaterals. It is described in detail in the Premium Debt section.

Spokes can call the `draw()` function, specifying the requested amount and the recipient. The function validates that the requested amount plus current drawn amount does not exceed the `drawCap` for the Spoke. Drawn shares are minted by dividing the requested amount by the current drawn index, and rounding up. Calls to `draw()` can fail if not enough liquidity is available. Debt can be repaid with the `restore()` function which allows specifying both a drawn debt amount and premium debt amount to repay, and pulls the required funds from the address specified.

It is important to note that the Hub does not perform any solvency checks on Spokes or users. Each Spoke is responsible for ensuring that its users maintain sufficient collateralization. The Hub is therefore trusting the Spokes to manage user solvency properly and maintain the overall health of the system. Only trusted Spokes should be given access to the Hub's liquidity.

2.2.1.3 Assets

Supported assets are non-rebasing ERC20 tokens, without transfer fees and hooks. An asset can be registered in the Hub through the permissioned `addAsset()` function. Registering an asset assigns it an `assetId`, and the same underlying token can be listed multiple times under different `assetIds`. The accounting for every listing of the same underlying will be kept separately. For every asset an interest rate strategy and interest rate data are configured. Each asset also has its own liquidity fee and fee receiver, described more in depth in the Fees section.

The permissioned `addSpoke()` allows enabling a Spoke to add and draw liquidity for an asset in the Hub. Each Spoke-Asset pair is configured with an `addCap` and a `drawCap` to limit the total amount of assets that can be added or drawn by the Spoke. This allows the Hub to manage risk across multiple Spokes.

2.2.1.4 Swept Liquidity

Every asset potentially has a `reinvestmentController` address. This is a special role that can allocate the liquidity of the asset to external products without minting debt shares and therefore accruing interest. The `reinvestmentController` can use `sweep()` to take liquidity from the Hub, and can give it back with `reclaim()`. Yield potentially generated by swept liquidity is opaque to the Hub, how the yield is attributed is at the discretion of the `reinvestmentController`. Swept liquidity does not contribute to the utilization factor used to compute the interest rate, for that purpose it is treated as available liquidity.

2.2.1.5 Deficit

Deficit, that is bad debt incurred by a Spoke, is tracked at the Hub level. The deficit amount is included in the total asset amount, and therefore still contributes to the value of supply shares, which is therefore non-decreasing. When a Spoke incurs a deficit, it reports this deficit to the Hub using `reportDeficit()`. The Hub then increases the total amount of assets in deficit for that asset. The deficit can be reduced with `eliminateDeficit()`. This function must be called from a Spoke and burns the amount of added shares by that Spoke to cover the deficit. It is assumed that an insurance mechanism is put in place to cover deficits when they occur.

2.2.2 Spokes

Spokes are contracts that serve as gateways for user interactions with the protocol. Each Spoke is deployed as an upgradeable proxy that points to the `SpokeInstance` implementation contract.

Users are expected to interact with the protocol through Spokes. Spokes are responsible for ensuring user-level solvency. As Spokes keep track of individual user positions, they enforce collateralization requirements and manage liquidations. They derive a health factor for each user based on their collateral balances and debt balances. Through the `LiquidationLogic` library invoked by `liquidationCall()`, Spokes allow liquidators to liquidate unhealthy users.

2.2.2.1 Reserves

Each Spoke maintains a list of reserves, where each reserve corresponds to an `(assetId,hub)` pair. Remapping `assetId`s to `reserveId`s allows Spokes to support assets from different Hubs. Every asset in a Hub can only be listed once as a reserve in the Spoke, but the same underlying token could be listed multiple times in a Spoke as different assets in the same Hub, or in different Hubs.

The reserve configuration includes whether it is in the `paused` or `frozen` state, where:

- `frozen`: supplying and borrowing are disabled; the reserve can still be disabled as collateral. Liquidations are not affected by freezing the collateral or debt reserve.
- `paused`: supplying, withdrawing, borrowing, and repaying are disabled, the reserve cannot be enabled/disabled as collateral. Liquidation with that reserve as collateral or debt are also disabled.

The reserve configuration also includes the `collateralRisk`, better described in section Premium Debt, and *dynamic reserve configurations*: the `collateralFactor`, the `maxLiquidationBonus`, and the `liquidationFee`.

Dynamic reserve configuration is a novel Aave V4 mechanism to update the parameters of a reserve without affecting existing loans that use that reserve. It allows the protocol to update collateral factors and liquidation parameters without negatively impacting existing positions right away. Thus, when reducing the collateral factor of a reserve, existing borrowers cannot be liquidated and have time to adjust their positions accordingly. Actions that reduce the user's health factor (withdraw, borrow) will always use the latest configuration when verifying the user's solvency but actions that increase the health factor (repay, supply) will use the configuration key of the user position to allow the user to improve their health factor even if the current configuration would make their position liquidatable.

In the Spoke's reserve data, a mapping is maintained between an integer `configKey` and a dynamic reserve configuration. When the configuration of a reserve is updated, the `configKey` is incremented

and the new configuration values are stored under the new key. For every user, a `configKey` is stored in their position, and it is used when calculating a user's health factor or determining liquidation parameters. The configuration key for all collaterals of a user are updated whenever they take actions that can worsen their health, such as `withdraw()`, `borrow()` or an asset is removed from the collateral list with `setUsingAsCollateral()`. When a reserve is added as collateral, the latest configuration for that reserve is used.

The user or its position manager can decide to `updateUserDynamicConfig()` to the latest configuration key for all collaterals. Authorized callers can also update any user's dynamic configuration if needed. Note that updating the dynamic configuration can only be done for all the user's collaterals and not individually.

The Spoke authority can also update any existing configuration with `updateDynamicReserveConfig()`. This action potentially worsens the health of existing positions, making them liquidatable.

2.2.2.2 Supplying liquidity

Users (or their authorized Position Managers) can supply liquidity to a reserve (therefore to an asset in a Hub) by using the `supply()` function. The supplied amount is deposited in the Hub of the specified `reserveId`, through `Hub.add()`, which pulls the underlying amount from the user's balance. The Hub operation increases the Spoke's share in the Hub, and the new amount of shares minted is credited to the user. Therefore, for every reserve, the Spoke holds shares of the corresponding Hub asset, and in the Spoke the shares are credited to individual users.

Users (or their authorized Position Managers) can withdraw supplied reserves through the Spoke's `withdraw()` function. The requested amount (or at most the user's balance) of a reserve is removed from the Hub, and the amount of shares burned for this operation is subtracted from the user's balance. Supplied reserves can be used as collateral for borrowing, therefore a collateralization check is performed when withdrawing a reserve that is used as collateral. If the withdrawn reserve is used as collateral, the user's risk premium is recomputed after the withdrawal. The withdrawal can also fail in case not enough liquidity is available in the Hub.

2.2.2.3 Borrowing

Spokes allow users (and Position Managers) to borrow liquidity available in the Hub, through the `borrow()` function. The Spoke draws the requested reserve amount from the Hub, through `Hub.draw()`, which increases the Spoke *drawn shares* balance for that asset. These drawn shares are in turn credited to the borrowing user. To ensure solvency of borrowers, the user's health is validated after borrowing. The risk premium of the borrower is also updated. Borrowing can revert if not enough liquidity is available in the Hub.

Debt can be repaid through function `repay()`. Each user has two separate debt balances, the *drawn* debt and the *premium* debt. The *drawn* debt is due to the accrual of the borrowed amount according to the interest rate calculated in the Hub. Additionally, the accrued amount is multiplied by the *risk premium* and goes to constitute the premium debt. When repaying, first the premium debt of a user is reduced, and then the drawn debt.

2.2.2.4 Position Managers

Users can enable `positionManager`'s to manage their positions on their behalf. The `NativeTokenGateway` and the `SignatureGateway` are two instances of position managers. The `NativeTokenGateway` allows users to supply and withdraw the native chain token (e.g., ETH) by wrapping and unwrapping it to its corresponding wrapped token (e.g., WETH). The `SignatureGateway` allows users to perform gasless operations by signing messages off-chain that are then relayed on-chain by a third party. Position managers must be approved by the user to manage their positions through `setUserPositionManager()`. Spokes decide which position managers are available to users through `updatePositionManager()`.

2.2.3 Collateralization

Aave V4's borrowing is overcollateralized to ensure the solvency of the system. The borrowing power of a user is the sum of the value of their supplied collaterals multiplied by the collateral factors of the collaterals. The health factor of a user is then calculated as follows:

$$\mathcal{HF} = \frac{\sum_i (CF_i \cdot V_i)}{D_v}$$

where CF_i is the collateral factor of asset i, V_i is the value of the supplied amount of asset i, and D_v is the total value of the user's debt.

A Spoke can configure the collateral factor for each of its reserves. If a user's health score falls below 1, they become liquidatable.

A user can decide which of their supplied assets to use as collateral with `setUsingAsCollateral()`. Only assets used as collateral contribute to the user's borrowing power and can be seized during liquidation.

2.2.4 Position Status Map

A user's positions within a Spoke are stored in a `PositionStatusMap`. For each `reserveId`, two bits are used to indicate whether the reserve is being used as collateral and/or borrowed by the user. One word can store information for up to 128 reserves. The first 7 bits of the `reserveId` are used to index the bit within the word, while the remaining bits specify the `bucketId` that holds the word for the corresponding reserve.

Whenever a user adds a reserve as collateral or borrows from a reserve, the corresponding bits in the `PositionStatusMap` are set with `setBorrowing()` and `setUsingAsCollateral()`. When a user removes a reserve as collateral or repays all their debt from a reserve, the corresponding bits are cleared. The Spoke uses this map to efficiently iterate over a user's collaterals and debts when calculating their health factor and risk premium or when updating the premium debt for every borrow position.

2.2.5 Premium Debt

Aave V4 introduces premium debt, which represents an additional portion of a borrower's debt that accrues interest based on the risk of their collateral. Borrowers supplying riskier collateral will accrue higher premium debt, which increases their total debt. Aave V4 updates a borrower's premium debt whenever the premium risk could increase, such as when a reserve is no longer used as collateral or when a collateral is withdrawn. As this operation can be costly, the premiums are not updated when the risk decreases (e.g., when a borrower supplies more collateral or repays debt). Borrowers should themselves call `updateUserRiskPremium()` to update their premium debt when the risk decreases.

Note that the premium debt is updated during liquidations as the risk premium change could increase if the liquidator liquidates a collateral that is configured as safe, while leaving riskier collaterals.

The risk premium of the borrower is a number, and is determined by sorting the collaterals from least risky to most risky based on their `collateralRisk` parameter. Then, starting from the least risky collateral, the system sums up the value of the collaterals multiplied by their respective `collateralRisk` until the total value of the collaterals reaches the total debt of the borrower. This weighted sum is then divided by the total debt to yield the risk premium. In this sense, the risk premium can be thought of as an average of the collateral risk factors, weighted by collateral amount, truncating the collaterals where the debt is fully covered:

$$riskPremium = \frac{\sum_i (V_i \cdot r_i)}{D_v}$$

subject to

$$\sum_i V_i \leq D_v$$

and

$$r_i \leq r_{i+1}$$

The last collateral included in the sum may be only partially included to ensure that the total collateral value matches the debt value D_v exactly.

The risk premium of a user is used to calculate the extra amount of interest that the user is charged. The risk premium is a factor that is multiplied to the base interest, and the result is the premium interest charged to the user.

This is implemented by minting *premium shares* to the user, proportionally to their drawn shares and their risk premium. The premium shares accrue interest at the same interest rate as drawn shares, but only the interest of premium shares has to be repaid and not the principal, so when they are issued, their current value is recorded (this is the `premiumOffset`), so that it can be later deducted from the future value of the shares. The difference is the *accrued premium*.

$$premiumShares_j = drawnShares_j \cdot RiskPremium$$

Every time the drawn shares of the user change (on `borrow()`, `repay()`, or liquidations), the risk premium of the user changes (disable collateral, `withdraw()`), or the premium debt is repaid (`repay()` or liquidations), the accounting of the premium debt is updated. The risk premium in principle is continuously changing, because the debt and collateral continuously change because of interest, and their values change because of price changes, however these changes are ignored as they are considered minor. In case a user's risk premium in use becomes considerably different to the risk premium that would be computed, it can be refreshed by a restricted role (or by the user themselves) through `updateUserRiskPremium()`.

The amount of premium shares changes when the drawn shares change, or the risk premium of the user changes. The premium debt accrued (`accruedPremium`) by the old premium shares is therefore "stashed" in the `realizedPremium` accumulator, and new `premiumShares` and `premiumOffset` amounts are computed. Likewise, if only premium debt is repaid (and drawn shares and risk premium do not change), the accounting must also be updated: the new realized premium is computed, the repaid amount is deducted from it, and the premium offset and shares are updated.

$$accruedPremium = premiumShares \cdot drawnIndex - premiumOffset$$

$$realizedPremium_{n+1} = realizedPremium_n + accruedPremium$$

Premium debt is always repaid before drawn debt (which accrues interest) in repayment operations (liquidations and repayments).

2.2.5.1 Premium delta

The accounting of premium debt is performed at the user level in the Spoke, and at the Spoke level in the Hub. When a user is updated in a Spoke, the change in their balances is forwarded to the Hub as signed integer deltas, which are applied to the overall Spoke balances maintained in the Hub.

In conclusion, a borrower's total debt D is the sum of their drawn debt, the realized premium, and the accrued premium debt of each of their borrowing positions.

2.2.6 Interest Model

The interest rate strategy for an asset is defined in the Hub. The interest rate strategy determines the interest rates for drawn amounts based on the utilization rate of the asset in the Hub. The utilization rate is defined as the ratio of total borrowed amount to the total supplied amount of the asset. As the utilization rate increases, the borrow interest rate increases according to the strategy defined for the asset.

The `AssetInterestRateStrategy` contract implements a piecewise linear model with two slopes. The strategy is defined by four parameters: `optimalUsageRatio`, `baseVariableBorrowRate`, `variableRateSlope1`, and `variableRateSlope2`.

When the utilization rate is below the optimal utilization rate, the borrow interest rate increases linearly from the base variable borrow rate to a rate determined by the first slope. When the utilization rate exceeds the optimal utilization rate, the borrow interest rate increases linearly according to the steeper second slope.

The maximum interest rate is capped at 1000% in the `AssetInterestRateStrategy` strategy, and is determined by

$$maxInterestRate = baseVariableBorrowRate + variableRateSlope1 + variableRateSlope2$$

`AssetInterestRateStrategy` is used by the Hub to define interest strategies for its assets with custom parameters set through restricted Hub function `setInterestRateData()`. Other interest rate strategies can be set with `updateAssetConfig()`.

2.2.7 Liquidations

When a user's health factor falls below 1, their position becomes eligible for liquidation. Liquidators can repay a portion or all of the user's debt in exchange for a corresponding amount of a user's collateral, plus a liquidation bonus. Liquidators are limited in the amount they can liquidate based on the `targetHealthFactor` parameter. A liquidator cannot liquidate more than what would bring the user's health factor above the target.

The liquidation bonus is controlled by three parameters: `maxLiquidationBonus` which varies per reserve, and `healthFactorForMaxBonus` and `liquidationBonusFactor` which are global per Spoke. The bonus starts increasing once a user's health factor drops below the liquidation threshold and begins at `maxLiquidationBonus * liquidationBonusFactor`. From there, it rises linearly as the health factor declines, reaching `maxLiquidationBonus` when the health factor is equal to or below `healthFactorForMaxBonus`.

Liquidations cannot leave less than `DUST_LIQUIDATION_THRESHOLD` amount of debt or collateral in a position, which is defined in terms of value (dollars) as $1000. A requested liquidation amount leaving less debt than this will be increased to the total debt amount, and likewise if the liquidated collateral would be below the threshold after the liquidation, the full collateral amount is liquidated instead. Amounts below `DUST_LIQUIDATION_THRESHOLD` can be left in the following conditions:

- A dust amount of collateral can be left after the liquidation if the full debt amount is liquidated.
- A dust amount of debt can be left if the initial requested liquidated debt would leave the corresponding collateral amount below the threshold. In such a case, the full collateral is liquidated, potentially leaving debt below the dust threshold.

2.2.8 Oracles

The Spoke uses the `AaveOracle` contract to query reserve prices. The AaveOracle contains a mapping from reserves to *price sources*, which are contracts exposing the Chainlink *AggregatorV3Interface* (`latestRoundData()`). Only the `answer` return value of the call to the price source is used, the timestamp of the answer is not validated. It is assumed that the price sources themselves will perform any necessary sanitization and validation.

2.2.9 Fees

The protocol collects fees from the interest paid by borrowers to suppliers and from liquidations.

For each asset in a Hub, every time interest accrues, some fee shares are given to the `feeReceiver` of the specific asset. The amount of fee shares is determined by the `liquidityFee` parameter of the asset. The fee shares accrue interest at the same rate as the supply shares.

Liquidation fees are collected during liquidations. A portion of the collateral seized during a liquidation is allocated to the `feeReceiver` of the collateral asset through `payFeeShares()`. The amount of collateral allocated as fee is determined by the `liquidationFee` parameter of the collateral asset. The seized collateral allocated as fee is converted to supply shares at the current share price and given to the `feeReceiver`.

The `feeReceiver` is expected to typically be a `TreasurySpoke` that receives fees and cannot `borrow()`, `withdraw()` or perform a `liquidationCall()`.

2.2.10 Hub & Spoke configurations

The Spoke authority can set global configurations for the Spoke and for its reserves. Additionally, the authority can be transferred to the `SpokeConfigurator` which can only call configuration functions on the Spoke, and it is owned by an `owner` address.

In a Spoke, a reserve can be frozen or paused.

A `HubConfigurator` can freeze or pause an asset:

- Frozen: The supply and borrows caps for each Spoke that is enabled for that asset are set to zero. Operations on the asset are therefore constrained to reduce existing supply and borrow positions.
- Paused: All reserves of this asset in all Spokes are set to inactive. `add()`, `draw()`, `repay()`, `withdraw()`, `reportDeficit()`, `eliminateDeficit()`, `payFeeShares()`, `refreshPremium()`, and `transferShares()` are therefore disabled.

A specific Spoke can also be frozen or paused for all its reserves coming from the Hub:

- Frozen: The supply and borrow caps for all reserves of that Spoke that are linked to an asset from the Hub are set to zero. Operations on these reserves are therefore constrained to reduce existing supply and borrow positions.
- Paused: All reserves of that Spoke that are linked to an asset from the Hub are set to inactive. `add()`, `draw()`, `repay()`, `withdraw()`, `reportDeficit()`, `eliminateDeficit()`, `payFeeShares()`, `refreshPremium()` and `transferShares()` are therefore disabled for these reserves.

2.2.11 AccessManagerEnumerable

The `AccessManagerEnumerable` is expected to be set as the Authority contract of Hub, Spoke, HubConfigurator and SpokeConfigurator. The contract extends OpenZeppelin's `AccessManager` with enumeration capabilities, allowing a limited, partial enumeration of the permissions configuration to be queried through view functions.

The contract maintains seven `EnumerableSet` data structures that shadow the base `AccessManager` state: a set of all role identifiers, a set of all admin roles, per-role member sets, per-admin-role managed role sets, per-role target contract sets, and per-role per-target function selector sets. These sets are kept in sync with the parent contract by overriding the internal functions `_setRoleAdmin()`, `_setRoleGuardian()`, `_grantRole()`, `_revokeRole()`, and `_setTargetFunctionRole()`. `PUBLIC_ROLE` and `ADMIN_ROLE` are excluded from the enumerable role set.

2.3 Trust Model

The following roles exist in the protocol:

- `Hub Authority`: The Hub has `restricted` functionality, an `authority` contract defines by which callers these functions can be accessed. The `authority` contract can therefore give permission to configure the Hub and its assets, adding Spokes, setting interest rate strategies, liquidity fee, supply and borrow caps for Spokes, and pausing or freezing assets, and setting the `feeReceiver` and `reinvestmentController`. The `authority` contract is expected to be correctly configured, in a way to limit privileged actions to **fully trusted** roles.

- `Spoke Authority`: The Spokes have `restricted` functionality, an `authority` contract defines by which callers these functions can be accessed. The `authority` contract can therefore give permission to configure the Spokes and their reserves, including setting collateral factors, collateral risk, liquidation parameters such as the max liquidation bonus and fee, and pausing or freezing reserves. The roles that can access these functionalities are **fully trusted**.
- `Spoke Proxy Admin`: **fully trusted**, can upgrade the Spoke implementation to a new version if the Spoke is deployed as an upgradeable proxy.
- **Spokes**: if Spokes are added in a Hub which are not instances of the `Spoke.sol` under review, they are **fully trusted** by the protocol.
- `SpokeConfigurator Owner`: **fully trusted**, can use the SpokeConfigurator to configure Spokes.
- `HubConfigurator Owner`: **fully trusted**, can use the HubConfigurator to configure Hubs.
- `Fee Receivers`: receive fees collected by the protocol. Expected to be instances of the `TreasurySpoke` contract. Otherwise **fully trusted**.
- `Reinvestment Controller`: fully trusted, manages the allocation of assets swept from the Hub to external products and can reclaim them when needed.
- `Position Managers`: fully trusted, can manage user positions on their behalf, including supplying, withdrawing, borrowing, repaying, and liquidating positions. They must be approved by the user to act on their behalf and by the Spoke to be allowed as a position manager.
- `Oracles`: fully trusted to provide accurate prices on-chain.
- `Users`: untrusted, can supply, borrow, repay, withdraw and liquidate positions through Spokes.

Tokens added to Hubs are considered to behave as standard ERC20 tokens, they are non-rebasing, they do not have hooks on transfers, they are not double-entrypoint tokens, and do not implement fees on transfers.

Spokes have full control over the premium drawn values in the Hub. Therefore, Hubs fully trust Spokes to truthfully report premium debt updates to the Hub when a user's risk premium changes. In the worst-case scenario, a malicious Spoke could inflate the amount of premium shares for a given asset in the Hub, which would inflate the value of added shares after accrual allowing the malicious Spoke to drain the Hub liquidity for that asset. Therefore, users using a Spoke must not only trust the Hub but also all other Spokes connected to that Hub that have at least one common asset with the Spoke they are using.

2.3.1 Changes in Version 4

Version 4 introduces several changes including:

- Underlying assets in the Hub are now unique, i.e. the same underlying token cannot be added multiple times as different assets in the same Hub.
- `supply()` and `repay()` in the Spoke now pull the underlying tokens from the user, and push them to the Hub, instead of the Hub pulling them directly from the user.
- A new flag `paused` was introduced for Spokes to allow for a more granular pausing mechanism instead of only having `active/inactive`. The `paused` state is similar to `inactive`, but allows `refreshPremium()` to succeed, which is part of the liquidation flow of unrelated reserves in the Spoke.
- The `SpokeConfigurator` holds a reserve limit per Spoke that limits the number of reserves that can be added to a Spoke.
- Dynamic configuration keys are now `uint24` instead of `uint16`. Moreover, dynamic configs are no longer stored in a ring buffer but are capped to `uint24.max`.
- On liquidations where deficit is reported, the risk premium is now updated to 0.
- Fee shares are no longer minted in `accrue` but in a separate function `mintFeeShares` which is permissioned.

- Liquidators can now choose to receive collateral shares instead of the underlying asset during liquidations.
- `ReserveFlagMap` is now used to store Reserve states in Spokes.
- Premium debt calculations have been optimized by no longer tracking the realized premium fee separately but including it in the premium offset, thus making the premium offset signed.

2.3.2 Changes in Version 6

In Version 6, `AccessManagerEnumerable` was taken into scope and reviewed. No other changes were reviewed at this point.

2.3.3 Changes in Version 7

Since Version 6 only reviewed `AccessManagerEnumerable`, the changes listed below span the whole period since Version 5, rather than only the changes introduced since Version 6.

Version 7 introduces several changes including:

- In `Hub`, the fund flow in `reclaim()` has been modified to mirror the flow in `add()` and `restore()`: Funds are directly sent to the Hub before `reclaim()` is called.
- In `Hub`, function `eliminateDeficit()` is `restricted`. Only whitelisted spokes can now call it.
- `MAX_USER_RESERVES_LIMIT` has been introduced to limit the maximum number of collaterals and borrows a user account can have.
- In `Spoke`, the `paused` flag has been renamed to `halted`.
- `Spoke` derives from `Extsload`, which provides external view functions to access arbitrary storage slots of the contract.
- The liquidation logic has been modified to reduce collateral and debt of a user by whole share amounts, minimizing rounding losses.

2.3.4 Changes in Version 8

Version 8 introduces several changes including:

- In `LiquidationLogic`, the rounding direction for the liquidation fee computation has been changed to round up against the liquidator when splitting the collateral between the liquidator and the fee receiver.
- In `AssetInterestRateStrategy`, parameters, errors, and events have been renamed to better reflect their purpose (e.g., `variableRateSlope1` to `rateGrowthBeforeOptimal`). The condition enforcing `slope1 <= slope2` has been removed from `_setInterestRateData()`.
- `AggregatorV3Interface` has been removed. Subsequently, `AaveOracle` has been reworked to use `IPriceFeed` instead.
- `UnitPriceFeed` was removed.

2.3.5 Changes in Version 9

Version 9 introduces several changes including:

- `Hub` has been made upgradable.
- A new `PositionManagerIntentBase` class has been introduced to handle `IntentConsumer` inheritance, decoupling it from `PositionManagerBase`.
- `renouncePositionManagerRole()` is only callable by the PositionManager owner on registered Spokes.

- `AccessManagerEnumerable` now tracks role labels.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

- Correctness: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	1

- AccessManagerEnumerable Reports Members as Active Before grantDelay Elapses Acknowledged

5.1 `AccessManagerEnumerable` Reports Members as Active Before grantDelay Elapses

Correctness Low Version 6 Acknowledged

CS-AAVE4-026

`_grantRole()` in `AccessManagerEnumerable` adds members to the `_roleMemberSet` immediately, regardless of the role's `grantDelay`. Meanwhile, `hasRole()` returns `false` until the delay elapses. This means `getRoleMemberCount()` and `getRoleMembers()` report members who do not yet have active permissions.

Systems relying on these enumerable views will incorrectly assume those members are active during the grant delay window.

Acknowledged:

Aave Labs acknowledges the findings and states:

> The *AccessManagerEnumerable* contract does not support delayed role grants. Roles are therefore enumerated immediately upon being granted, even if they are configured with a delay and are not yet effective. This behavior will be documented so that integrators are aware of this limitation.

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Open Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	2

- Phantom Members in AccessManagerEnumerable Role Enumeration Code Corrected
- Array.sort() Can Revert Due to EVM Stack Limit Code Corrected

Informational Findings	4

- assetId Cannot Be Determined From Underlying Code Corrected
- Ambiguous Revert Reasons Code Corrected
- Inconsistent Variable Names Code Corrected
- No On-Chain Reverse Mapping for reserveId Code Corrected

6.1 Phantom Members in `AccessManagerEnumerable` Role Enumeration

Correctness Low Version 6 Code Corrected

CS-AAVE4-029

`_revokeRole` override calls `_trackRoleMember(roleId, account, !revoked)`. Thus, when `renounceRole` is called by an account that does not hold the role, the parent returns `revoked = false`, so the negation is passed to `_trackRoleMember`, adding the caller as a phantom member in `_roleMemberSet`.

Since `renounceRole` is permissionless, any address can add itself to the enumerable member list of any role. On-chain access control (`hasRole`) is unaffected, but systems relying on `getRoleMembers` will observe incorrect memberships.

Code corrected:

In Version 7, the tracking logic on role revocations has been fixed to no longer add the caller to `_roleMemberSet` when no revocation has happened.

6.2 `Array.sort()` Can Revert Due to EVM Stack Limit

Correctness Low Version 1 Code Corrected

CS-AAVE4-012

When computing the risk premium for a user, the Spoke sorts the collateral assets provided by the user using `Array.sort()` from least to most risky. When 170 or more collateral assets are sorted, the sorting algorithm (quicksort) can revert because the EVM limit of 1024 stack elements is reached. Quicksort is a recursive algorithm, and every internal recursive call increases the stack utilization. The revert happens with 170 elements when the list before being processed by quicksort is initially already in sorted order (ascending risk), or inversely sorted (descending risk), as these conditions provide the worst-case complexity for this variant of quicksort: `O(n^2)` operations, and n recursive calls. A randomly sorted list requires `O(log(n))` recursive calls.

In practice, enabling more than 170 collaterals could mean that a position can no longer be liquidated as `_calculateUserAccountData()` reverts in `liquidationCall()`.

Specification changed:

In Version 3 Aave Labs modified the Spoke Configurator to set a reserve limit on Spokes. Additionally, in Version 7, the Spoke contract itself has been modified to include a limit on the number of collaterals and borrows for each user.

6.3 `assetId` Cannot Be Determined From Underlying

Informational Version 3 Code Corrected

CS-AAVE4-024

The Hub contract uses an internal mapping from `assetId` to `Asset` which contains the address of the underlying asset. However, there is no reverse mapping from the underlying asset address to the corresponding `assetId`. This might make it difficult for third-party integrators to determine the `assetId` associated with a given underlying asset address.

Acknowledged:

In Version 7 the `getAssetId()` external method has been added to return the `assetId` from the underlying token address.

6.4 Ambiguous Revert Reasons

Informational Version 1 Code Corrected

CS-AAVE4-013

- In `_validateReportDeficit()` and `_validateRestore()`, the `drawnAmount` and the `premiumAmount` are verified to be respectively less than or equal to `drawn` and `premium`. However, both `require` calls will revert with the same error `SurplusDeficitReported`. The error has an argument, which could represent either drawn or premium debt,

but with no indication on which one it is. Therefore, which condition failed in case of a revert, and the meaning of the error's argument could be ambiguous.

- In `SignatureGateway.updateUserDynamicConfigWithSig()`, `block.timestamp <= deadline` is checked to ensure that the signature is still valid. However, if this condition fails, the function will revert with `InvalidSignature` error. This error name does not clearly indicate the exact reason for the revert (i.e. an expired signature).

Code corrected:

The first issue was addressed by introducing two distinct errors: `SurplusDrawnDeficitReported` and `SurplusPremiumRayDeficitReported` in `_validateReportDeficit()` and `_validateRestore()`.

Aave Labs provided the following comment:

> The first issue has been fixed. For the second, introducing a dedicated error for expired signatures is not considered necessary, as an expired signature is still an invalid signature in all cases.

6.5 Inconsistent Variable Names

Informational **Version 1** **Code Corrected**

CS-AAVE4-016

- In `SignatureGateway`, some functions use `onBehalfOf` as parameter name (e.g. `setUsingAsCollateralWithSig()`), while others use `user` (e.g. `setSelfAsUserPositionManagerWithSig()`) to refer to the same address.

Acknowledged:

In Version 7, all `SignatureGateway` parameters use the `onBehalfOf` name to specify the target of the action.

6.6 No On-Chain Reverse Mapping for `reserveId`

Informational **Version 1** **Code Corrected**

CS-AAVE4-019

In the Spoke, it is possible to obtain the `assetId` and `Hub` address from a `reserveId`, but it is difficult to check if a given `(assetId, hub)` pair is already listed in the Spoke, and with what `reserveId`, without iterating through all reserves.

An inverse mapping could be made available by expanding the internal `_reserveExists` function for this purpose.

Acknowledged:

In Version 7, the `getReserveId()` method was added which returns the `reserveId` when passed the `hub` and `assetId`, or reverts if it is not listed.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 `reportDeficit` and `eliminateDeficit` Return Rounded Deficit Amounts

Informational **Version 8** **Acknowledged**

CS-AAVE4-031

`reportDeficit` and `eliminateDeficit` both return the deficit amount converted from RAY precision to asset units via `fromRayUp()`, which rounds up. This means the returned value can be slightly larger than the actual deficit eliminated internally (stored as `deficitAmountRay`).

Spokes relying on the returned value to update their own accounting or to display the eliminated deficit to users would work with an overstated amount. The full RAY-precision value (`deficitAmountRay`) is available via the emitted events (`ReportDeficit` and `EliminateDeficit`), but not directly from the return value.

Acknowledged:

Aave Labs acknowledges the findings and states:

> The implemented precision is consistent with the rest of the Hub functions, providing a standardized approach that reduces the likelihood of integration errors. Spokes requiring higher precision when validating the eliminated deficit can query the Hub for the deficit values in RAY both before and after the operation using the `getSpokeDeficitRay` function.

7.2 Uninitialized Hub and Spoke Return Incorrect Listing Info

Informational **Version 7** **Acknowledged**

CS-AAVE4-028

Method `isUnderlyingListed()` of *Hub*, when queried with `address(0)`, incorrectly returns `true` when queried before any underlying has been listed in the Hub (meaning that `AssetId == 0` is still unused).

Similarly, method `getReserveId()` of *Spoke* returns `0` when queried with `hub: address(0), assetId: 0` instead of reverting, if called before any reserve is listed in the Spoke (`reserveId == 0` is still unused).

Acknowledged:

Aave Labs acknowledges the findings and states:

> This behavior is considered acceptable, as no state-changing operations are possible while no assets or reserves are listed. The getters function correctly once at least one asset or reserve has been listed, which is the point at which the contract becomes operational.

7.3 `AccessManagerEnumerable` Maintains Unnecessary Storage for Target-Selector Role Tracking

Informational **Version 6** **Acknowledged**

CS-AAVE4-030

`_trackRoleTargetSelector()` in `AccessManagerEnumerable` reads the old role ID from a private mapping `_targetToSelectorToRole` before updating the enumerable sets. This mapping exists solely to retrieve the old value before it is overwritten. This is a redundancy since the parent `AccessManager` already exposes this data via `getTargetFunctionRole(target, selector)`.

Acknowledged:

Aave Labs acknowledges the findings and states:

> The logic and storage of this function are intentional, retained to preserve consistent behavior across all overridden functions. Each function of the contract follows the same pattern of executing the parent implementation first, followed by updating the additional tracking storage, keeping the implementation simple and consistent.

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Liquidation Gas Cost Is Possibly More Than Reward

Note Version 1

Liquidations must iterate through all the borrowed assets and collaterals of the liquidated user. For each asset, the process involves loading the asset configuration, querying an oracle, loading user balances, and querying the Hub to convert share balances to assets. Every iteration consumes some gas, and the number of iterations is only capped by the number of enabled reserves in the spoke.

Since Version 7, the number of iterations is bounded by `2 * MAX_ALLOWED_USER_RESERVES_LIMIT`. In practice, a borrower could enable `MAX_ALLOWED_USER_RESERVES_LIMIT` collaterals and `MAX_ALLOWED_USER_RESERVES_LIMIT` borrowable assets on their account to maximize the liquidation cost.

Moreover, the cost of each iteration as a portion of the liquidation reward is not bounded. There is no minimum debt size for a borrowed asset, therefore the liquidation incentive could be smaller than the liquidation gas cost. A borrower could split a position into several smaller ones to make it less attractive to liquidate.

The positions could be created when gas prices are low; however, liquidations often need to happen during times of high asset price volatility when gas prices are high. Therefore, the gas price for the borrowers is less than for the liquidators.

As an example, let us consider `MAX_ALLOWED_USER_RESERVES_LIMIT = 10`, such that a borrower has a position with 10 borrowed assets and 10 collateral assets. For simplicity, only 1 of the borrowed assets has a significant debt balance, and only one of the collaterals has a significant balance, and the liquidator therefore only needs to perform a single liquidation.

Let us assume gas prices are 1 Gwei, ETH price is $3000, liquidation bonus is 5%, and every asset enabled by the borrower costs 80'000 gas to iterate through.

The gas cost of the liquidation is therefore:

```
20 (assets enabled) * 80k (gas per asset) * 10^-9 (ETH per gas) * 3000 (USD p
er ETH) == $4.8
```

With a 5% liquidation bonus, debt smaller than ~`$100` becomes therefore unprofitable to liquidate.

Despite the asymmetry of gas prices between the time of creation of the position and the time of liquidation, the borrower uses considerably more gas to create the position than what is used to calculate their health. Creating the position involves setting storage slots for the first time, which is considerably more gas intensive than reading (cold) storage slots.

As a result, whenever the liquidation reward fails to cover the gas cost, no rational liquidator will act on the position, leaving it undercollateralized indefinitely as the debt keeps growing.

Acknowledged:

Aave Labs acknowledges the findings and states:

> Having liquidation rewards lower than the gas costs required to process a complex user position is considered an edge case. This risk can be continuously mitigated through protocol configuration by adjusting liquidation bonuses and limiting the maximum number of reserves that a user can include in a position. The gas costs incurred to build and maintain a highly fragmented multi-reserve position act as a natural economic deterrent, as they may exceed any potential benefit gained from attempting to avoid liquidation. Furthermore, in the unlikely event that a position becomes unprofitable for independent searchers to liquidate, the protocol can rely on subsidized liquidators to resolve these edge cases and preserve overall solvency.

8.2 Paused Assets Keep Accruing Interest

Note **Version 1**

In the `HubConfigurator` contract, `pauseAsset()` can be used to pause a specific asset in the Hub. This leads to all Spoke configs for this asset being set to inactive.

When an asset is paused, no actions can be performed with this asset in the Spokes, e.g., no new loans can be opened, no new deposits can be made, and no repayments can be performed. However, interest on existing loans continues to accrue. Users that have an open loan in such an asset cannot repay their loan, as repayments are disabled for paused assets.

Users are therefore forced to let their loan continue to accrue interest.

Typically, such a situation is not expected to last long, and interest accrual during negligible time frames is not a big issue. However, if an asset remains paused for a long time, this can lead to significant interest accrual on existing loans that users cannot repay.

Aave Labs confirms that this behavior is intentional, and states:

> Pausing an asset is not expected to be a long-lasting condition, and therefore the interest accrued during that period is not considered significant enough to cause harm to the protocol. By contrast, if the interest generated during the pause is deemed problematic, the Spoke owner can update the interest rate to mitigate its impact.

8.3 Share Value Can Grow Faster Than Interest Rate Because of realizedFees Accrual

Note **Version 5**

`realizedFees` is an amount of supplied asset that participates to liquidity, but is not backed by shares. If the `realizedFees` value is borrowed, it will accrue interest, which increases the value of added shares. It has been pointed out, during a review by Certora, that in extreme cases `realizedFees` are much larger than the total share value, which can cause the value of shares can significantly grow in value, beyond the normal growth caused by normal interest accrual.

This can happen in particular just after an asset has been listed, when a single depositor joining and leaving can significantly change the ratio between `realizedFees` and total assets, and when an asset is off-boarded, which causes assets to be withdrawn, potentially significantly increasing the ratio between `realizedFees` and total assets.

The increase in share price caused by this edge-case behavior does not cause problems internally in Aave V4, but might interact negatively with external integrations that assume small share prices.

The potentially unbounded share growth caused by `realizedFees` can be limited by converting `realizedFees` into shares often, through frequent calls to `mintFeeShares()` (permissioned), and by maintaining a small minimum deposit in every asset:

The triggering of the behavior can be roughly modeled as follows: A large debt of size C causes `realizedFees` f to accumulate over a time Δ_0. The added supply is then reduced from C to S and the debt is reduced from $C + Cr\Delta_0$ to $f + S$. The accumulated f then causes the share value to increase faster than the interest rate r.

$$f = lCr\Delta_0$$

`realizedFees` (f) are accumulated over a period Δ_0 with liquidity fee l and interest rate r from a large debt of C.

The debt is then mostly repaid, and withdrawn from the added supply. The added supply is now S and the debt is $f + S$. The growth of share value after a period Δ_1 can be modeled as:

$$\alpha S = S(1 + r\Delta_1) + f(r\Delta_1)$$

That is, the new total share value αS comes from the old value S plus its accrual, and from the accrual of debt on `realizedFees` f.

If one wants to limit the share value increase, for example by $\alpha < 2$, concrete values can be replaced for the variables. Assuming time intervals $\Delta_0 = \Delta_1 = 1day = 1/365$, interest rate is $r = 100\%$, initial debt value $C = \$100M$, and liquidity fee $l = 10\%$, then:

$$f = 10\% \cdot \$100M \cdot \tfrac{1}{365} \cdot 100\%$$

$$= \$27'397$$

Then constraining $\alpha < 2$:

$$\alpha = (1 + r\Delta_1) + \tfrac{f}{S} r\Delta_1 < 2$$

$$\Leftrightarrow$$

$$S > \tfrac{fr\Delta_1}{1 - r\Delta_1}$$

Again replacing concrete values:

$$S > \tfrac{27'397 * 100\% * 1/365}{1 - 100\% * 1/365} = \$75$$

So in this concrete example, based on a response time of around 1 day before `mintFeeShares()` is called after accumulating significant `realizedFees`, and a maximum debt of $100M, a minimum deposit of $75 is enough to prevent the share value from doubling.

Aave Labs acknowledges the note and states:

> We acknowledge this behavior, which will be prevented when we implement the Fee Minter contract to periodically call `mintFeeShares`.

8.4 Small Positions May Not Be Profitable to Liquidate Due to Rounding

Note **Version 5**

In liquidations, rounding is applied against the liquidator and in favor of the liquidated user. The liquidator receives slightly less collateral and needs to repay slightly more debt to ensure that the liquidated user's position improves. This means that for sufficiently small positions, the rounding loss can exceed the liquidation bonus, making the liquidation unprofitable.

The rounding loss has two components: collateral and debt:

- For collateral, the liquidator can avoid the rounding loss in `_liquidateCollateral()` by choosing appropriate parameters (`receiveShares`). Generally, collateral would get rounded in `_calculateCollateralToLiquidate()` which introduces an error of 1 collateral asset, due to `mulDiv` (converting debt to collateral assets) and 1 collateral share due to `previewAddByAsset` (converting collateral assets to collateral shares). In the subsequent example we consider for simplicity that the value of 1 collateral share is equal to 1 collateral asset.
- For debt, in the general case, 2 roundings arise from `rayMulUp` (converting drawn shares to debt assets) and `fromRayUp` (converting premium debt from RAY to asset units) in `_liquidateDebt()`.

Note that the exact roundings for collateral and debt depend on the liquidation path taken. The values above (2 debt assets, 2 collateral assets) are representative of the general case. For example, collateral rounding could be 0 if all the collateral shares of the user are liquidated, and the liquidator receives collateral shares.

We now use the above derived roundings to quantify the minimum position size below which liquidation becomes unprofitable.

Let the debt token have D_{debt} decimals and price P_{debt}, let the collateral token have D_{col} decimals and price P_{col}, let the liquidation bonus be b, and let the liquidation fee be f. The expected rounding loss is:

$$L_{\text{round}} = \frac{2}{10^{D_{\text{debt}}}} \cdot P_{\text{debt}} + \frac{2}{10^{D_{\text{col}}}} \cdot P_{\text{col}}$$

A liquidation is unprofitable when the rounding loss exceeds the effective bonus. Since the liquidation fee is taken on the bonus portion of the seized collateral, the liquidator's effective bonus is $b \cdot (1 - f)$. The condition becomes:

$$V \cdot b \cdot (1 - f) < L_{\text{round}}$$

$$\Leftrightarrow V < \frac{L_{\text{round}}}{b \cdot (1 - f)}$$

For example, with WBTC as debt (8 decimals, \$100,000), ETH as collateral (18 decimals, \$3,000), $b = 5\%$, and $f = 10\%$:

$$L_{\text{round}} = \frac{2}{10^{8}} \cdot 100000 + \frac{2}{10^{18}} \cdot 3000 \approx 0.002 \text{ USD}$$

$$V < \frac{0.002}{0.05 \cdot 0.9} = \frac{0.002}{0.045} \approx 0.044 \text{ USD}$$

The unprofitable-liquidation threshold is approximately 4.4 cents. The rounding loss is dominated by the low-decimal debt token (WBTC), while the 18-decimal collateral (ETH) contributes a negligible amount.

In Version 8, liquidation fees are rounded against the liquidator and in favor of the protocol via `mulDivUp`. This increases the collateral rounding from 2 to 3 assets. The impact is negligible for high-decimal collateral tokens (e.g., ETH with 18 decimals), but becomes meaningful for low-decimal collateral. For example, with ETH as debt (18 decimals, \$3,000) and WBTC as collateral (8 decimals, \$100,000), $b = 5\%$, and $f = 10\%$:

$$L_{\text{round}} = \frac{2}{10^{18}} \cdot 3000 + \frac{3}{10^{8}} \cdot 100000 = 0.003 \text{ USD}$$

$$V < \frac{0.003}{0.05 \cdot 0.9} = \frac{0.003}{0.045} \approx 0.067 \text{ USD}$$

With WBTC as collateral, the threshold rises to approximately 6.7 cents.

8.5 Type Bound Considerations

Note Version 1

- The `drawnIndex` has been modified from a `uint128` to a `uint120` in Version 2. This implies that for an asset at 30% APR (compounded daily) the `drawnIndex` will overflow after approximately 70 years. This is a theoretical limit and in practice it is unlikely that an asset will remain in the protocol for such a long time without any intervention.
- Premium debt is stored with added precision (RAY) which is 27 extra decimals of precision. As `premiumOffset` is stored as an `int200`, the possibility of overflow/underflow should be considered. The premium risk factor can be at most 1000%, i.e. 10 times more premium shares than drawn shares, so we can have the following:
    - 27 decimals precision takes log2(10^27) = 89.69 bits
    - 1000% (10x) premium risk takes up to log2(10) = 3.32 bits
    - sign of the integer takes 1 bit

We are left with 200 - 89.69 - 3.32 - 1 = 105.98 bits for the maximum amount of borrowed assets (`drawnShares * drawnIndex`). This corresponds to 31 decimal digits. For tokens with high circulating supply (in terms of balance amounts), such as PEPE, balances of the order of 10**31 are feasible. The total supply of PEPE is 4.2 * 10**32.

Aave Labs agrees that balances of the order of 10**31 are technically feasible in the Hub, but points out that with the current Spoke implementation, tokens with such a high balance are not feasible. Aave Labs states:

> While such values may be theoretically possible from the Hub's perspective, the Spoke only supports oracles with 8 decimals. As a result, the minimum representable price is 1e-8, implying that a balance of 10^31 would correspond to 10^23 USD, which is not considered a realistic or feasible amount.

- `drawCap` and `addCap` are represented as `uint40` which limits the cap to `2**40 - 1` units of asset which is about 1 trillion. For assets with low unit price the caps might not be able to grow enough to accommodate a significant amount of liquidity.